Naveed Muhammad Muneeb
CM903S

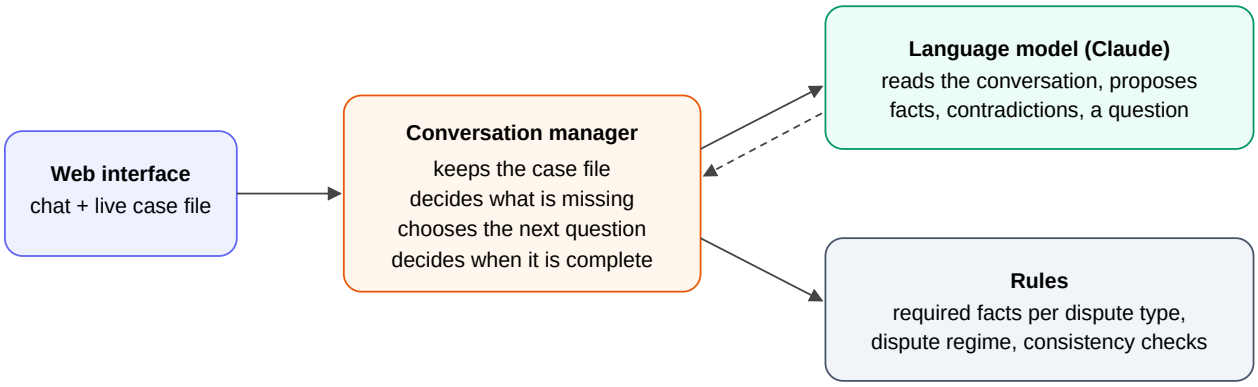
1. The problem

When a cardholder disputes a card payment, the bank first has to turn a complaint such as “someone charged me 180 euros at a shop I’ve never heard of” into a case it can act on. That needs a fixed set of facts that depends on the kind of dispute: merchant, amount, date, whether the card is still in the cardholder’s hands, whether anyone else was allowed to use it, whether the merchant was contacted. People rarely give all of these at once, state some vaguely, and sometimes contradict themselves. **Clerk** holds that conversation: it reads the cardholder’s own words, builds a structured case file, asks one plain question at a time for what is missing, and flags inconsistencies for a human reviewer. It never decides the dispute itself.

2. Design principle: the model extracts, the rules decide

A language model is good at what is hard to program, understanding free text in any wording or language, and unreliable at what must be trustworthy: never guessing a fact and applying fixed rules the same way every time. Clerk splits the work along that line. The model only *reads and proposes*; deterministic rules make every decision that matters.

3. Architecture



One turn. The cardholder’s message reaches the conversation manager, which sends the whole conversation to the model. The model returns a strictly structured proposal: every fact it can find, any contradictions it noticed (each quoting the conflicting words), and one suggested question. The manager merges the facts into the case file, applies the rules (which facts this dispute type still needs, which regulatory regime applies, hard consistency checks such as “card reported stolen but still in the cardholder’s possession”), and either asks the next question or closes the case. The model’s suggested question is used only if it asks for a fact that is actually still missing; otherwise a prepared question is asked. A conversation is capped at eight turns. The interface shows the case file filling in live, so a viewer sees missing facts shrink and flags appear as the conversation goes.

4. Prompt techniques

- **Persona:** the model acts as the intake assistant on a bank’s dispute desk, with explicit limits: no decisions, advice or promises.

- **Few-shot:** two worked examples, one showing unstated facts left empty and one showing an explicit “no” recorded as a no.
- **3C principle.** *Clarity:* definitions of the six dispute types, and the central rule “record a fact only if the cardholder stated it; never guess an amount, date or yes/no answer”. *Context:* the role and the filing date, so “yesterday” becomes a real date while “a while ago” stays empty. *Constraints:* one question per turn, plain language, and a fixed output structure enforced by the API.
- **Iterative refinement:** every prompt change was triggered by an observed failure and measured afterwards (§6, §7).

The model reasons internally before answering, with the depth set by an *effort* parameter. An attempt to make that reasoning visible as part of the output was refused by the model’s safety system and was dropped.

5. Parameters and error handling

The interface exposes the model’s thinking effort, the maximum response length and the turn cap. Because the main model does not accept sampling settings, a second model can be selected to demonstrate *temperature* and *top-p*. Low temperature suits extraction, since the same complaint should always give the same case file. Rate limits, timeouts and connection failures are retried automatically and then shown as a readable message with a retry button; empty input is rejected before anything is sent; a failed turn leaves the conversation unchanged.

6. Evaluation

Clerk was tested on 22 hand-written cases: every dispute type, one case per consistency rule, three “traps” in which a date is never given, foreign cards, direct debits, one complaint in Hungarian, one that changes its dispute type halfway, and two prompt-injection attempts. A scripted cardholder answers only what it is asked and says “I don’t know” otherwise, so any fact that appears without being stated was invented. Results are compared exactly against the expected case files, without a second model acting as judge.

	First run	**After one revision**
Facts extracted correctly	98.6%	**100%**
Invented facts	0	**0**
Correct regime / all required facts collected	20 / 18 of 22	**22 / 22 of 22**
Consistency flags found (precision / recall)	100% / 100%	100% / 100%

Reading every miss in the first run showed that questions about *optional* facts were using up the eight turns before required ones were asked, and that “a charge on my card” was not read as a card payment. One revision fixed both. Limits: one run per version, and the cases are self-written, not real complaints, so the result shows the design works rather than that it generalises.

7. Safety: prompt injection

Complaints often quote merchant e-mails, so the cardholder’s text is treated strictly as data: it is clearly separated from the instructions, the model can only fill a fixed structure, every decision is made by the rules, links and e-mail addresses are removed from anything the model writes, and attempts are flagged for the reviewer. Clerk was tested with 35 real attacks from Microsoft’s public LLMail-Inject challenge, each one chosen because it had beaten that challenge’s defended e-mail assistant. All 35 were detected in every run, no facts were invented, and

genuine complaint details stayed intact. In the first run the model copied an attacker’s e-mail address into its summary twice; after a prompt change and the address filter, that dropped to zero.

8. Limitations

Clerk only collects and flags; a person decides. Sessions are kept in memory and nothing is stored, so a real bank would add authentication, audit logs, data-protection-compliant storage, and a link to its own transaction records instead of trusting the cardholder’s figures. The regulatory regimes are simplified labels, not legal advice. Each turn takes about 5 to 25 seconds.

References. Abdelnabi, S., Fay, A., Salem, A., et al. (2025). *LLMail-Inject: A Dataset from a Realistic Adaptive Prompt Injection Challenge.* arXiv:2506.09956. · Anthropic, Claude API documentation.